

Clinova AI Triage Engine

Technical Architecture & Working Model Specification

A technical specification covering the hybrid intelligence of the Clinova patient chatbot. Details include the local Scikit-Learn machine learning classifier, Google Gemini API orchestration, Java Spring Boot REST interfaces, stateful database triage session mapping, and the resilient local no-crash fallback framework.

Author: Clinova Engineering Group

Platform: FastAPI (Python) & Spring Boot (Java)

Date: May 2026

1. Executive Summary & System Flow

The Clinova AI Triage Engine is a hybrid clinical reasoning service designed to process patient conversations, extract diagnostic symptoms, perform machine learning diagnostic classifications, and suggest immediate medical pathways. By decoupling clinical logic from the frontend and integrating a zero-dependency local fallback, the triage system maintains 100% availability even during Gemini API quota exhaustion or database server downtime.

1.1 Decoupled Microservice Architecture

The system operates across three sovereign layers of the Clinova clinical control suite:

*** React Patient Care Portal (Frontend):** A clean, modern chat interface where patients describe their symptoms and receive real-time, empathetic advice.

*** Java Spring Boot Backend (Orchestration):** Manages user state, verifies JWT authentication, stores stateful 'Triage Sessions' in the MySQL database, and schedules consultations upon triage completion.

*** FastAPI AI Engine (Reasoning Microservice):** A high-performance Python microservice that encapsulates local Machine Learning models and Google Gemini API connections.

1.2 Stateful Request Pipeline

Every patient interaction follows a stateful lifecycle:

1. The React Patient Care Portal posts the user message and patient vitals to the Spring Boot ChatController.
2. Spring Boot loads the active TriageSession for the authenticated patient and prepares a unified PythonChatRequest payload.
3. FastAPI parses the user message to extract confirmed/denied symptoms, feeds the confirmed list to the local Scikit-Learn model, and computes diagnostic probabilities.
4. If confidence is low, the chatbot conversational AI generates an empathetic follow-up tie-breaker question.
5. If confidence is high or maximum denials are reached, it finalizes the diagnosis, routes the patient to a specialist doctor, designs a custom 2-day diet, and returns status: TRIAGE_COMPLETE.
6. Spring Boot automatically saves a Consultation report and clears the active session state.

2. Core Working Model & Hybrid AI

2.1 Local Scikit-Learn Model (triage_model.pkl)

Deterministic clinical reasoning is achieved using a pre-trained local Scikit-Learn Machine Learning classifier. The model is lightweight, ultra-fast, and executes entirely inside your Python runtime environment without any internet connection.

* **Feature Input Space:** The model expects a binary vector of size 59, mapped to strict snake_case symptom keys loaded from 'symptoms_list.pkl' (e.g., 'acidity', 'vomiting', 'high_fever', 'joint_pain').

* **Vector Encoding:** Confirmed symptoms are set to 1.0; all other dimensions are set to 0.0.

* **Inference Runtime:** The vector is wrapped in a Pandas DataFrame to preserve clinical feature names, and fed to the model to predict class probabilities:

```
input_df = pd.DataFrame([input_data], columns=symptoms_list)
probabilities = model.predict_proba(input_df)[0]
```

* **Classification Classes:** Predicts the exact mathematical probability distribution for the top 3 matching medical conditions (e.g., Influenza, Gastrointestinal Disease, Migraine).

2.2 Google Gemini API Key Integration (gemini-2.5-flash)

While the ML model provides deterministic symptom classification, Google Gemini API provides advanced NLP reasoning. The engine integrates with the new 'google-genai' SDK to perform three specific clinical tasks:

* **1. Symptom & Denial Extraction:** Converts loose conversational language into clinical symptom keys. For example, if a patient says 'I have stomach ache but no headache', Gemini extracts stomach_pain to confirmed, and headache to denied_symptoms.

* **2. Empathetic Clinical Follow-ups:** If the ML model's confidence is under 65%, Gemini reads the clinical instructions, avoids confirmed/denied lists, and designs an empathetic tie-breaker question.

* **3. Personalized Nutrition Planning:** Upon triage completion, Gemini acts as a clinical nutritionist to generate a concise, customized 2-day diet plan tailored to the predicted disease and calculated BMI category.

3. Backend Integration & Resilient Fallbacks

3.1 Spring Boot REST Communication

The Java backend communicates with the Python FastAPI microservice using REST API calls. Spring Boot uses Java's RestTemplate with a strict 30-second read timeout to support slow AI generations under heavy server loads.

3.1.1 Request Payload Schema (PythonChatRequest)

```
{
  "user_message": "string",    // Latest user chat input
  "current_symptoms": ["string"], // Previously confirmed symptoms
  "denied_symptoms": ["string"], // Previously denied symptoms
  "chat_history": ["string"],   // Last 4 conversation lines for context
  "weight_kg": float,          // Patient weight (optional)
  "height_m": float            // Patient height (optional)
}
```

3.2 Resilient Local Fallback Engine (No-Crash Architecture)

In production environments, external API connections can hit limits (such as the Gemini Free Tier limit of 20 requests per day) or suffer internet downtime. The Clinova AI Triage Engine implements a highly robust fallback framework that automatically activates if the Gemini API key is exhausted or offline:

- * **Local Symptom Matching:** If Gemini fails, a local fuzzy keyword substring matcher maps the user message to clinical symptoms by checking word stems and exact match patterns.
- * **Local Follow-up Rules:** Automatically identifies unconfirmed common symptoms (like 'cough' or 'fever'), formats a natural nurse response, and asks the patient locally.
- * **Local Diet Planner:** Designs a clean, bulleted diet plan locally based on standard clinical rules (Hydration, digestible meals, BMI restrictions) without calling external servers.
- * **Java Backend Graceful Guard:** If the Python microservice is offline, the Spring Boot catch block shields the patient from raw system errors, returning an empathetic suggestion to reset the chat or consult a physician directly.

CLINICAL OBSERVATION:

The integration of local fallback matching with local ML classifiers ensures 100% diagnostic runtime uptime, making Clinova a robust, safe, and highly resilient clinical platform.